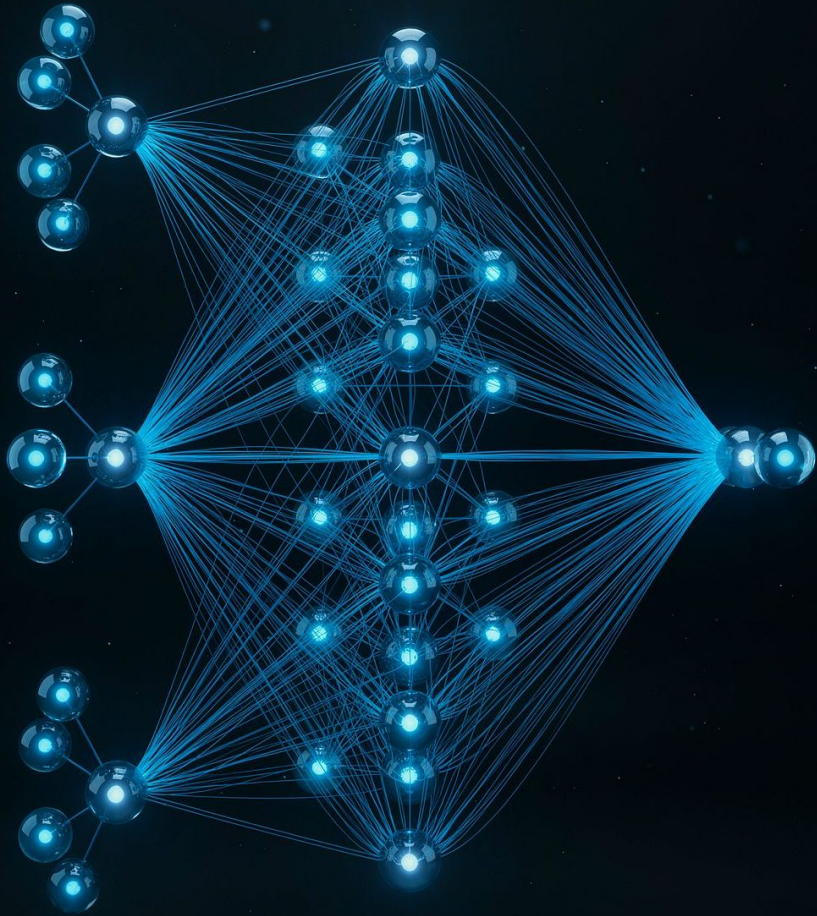




HuggingFace Transformers: API, Models & Fine-tuning Techniques

Day 5 — Module 9 & 10

Advanced Fine-Tuning Techniques and Applications — Instruction Tuning, Domain Adaptation, Unsloth, Deployment, and Evaluation Best Practices

Instructor: Chandrashekar Babu <training@chandrashekar.info>

<https://www.chandrashekar.info/> | <https://www.slashprog.com/>

Course Agenda — Day 5

MODULE 9

Advanced Fine-Tuning

01

Instruction Fine-Tuning and Chat Models

Dataset formats, chat templates, fine-tuning SmolLM2/Phi for instructions

02

Domain Adaptation Strategies

Continued pre-training, multi-task fine-tuning, data augmentation

03

Unsloth Framework

What is Unsloth, speed and memory optimisations, comparison with standard fine-tuning

04

Hands-on Lab

Instruction fine-tuning end-to-end with Unsloth

MODULE 10

Deployment & Best Practices

01

Model Evaluation

Intrinsic vs extrinsic evaluation, the `evaluate` library, custom metrics

02

Deployment Options

Gradio demos, ONNX/TorchScript export, HuggingFace Inference Endpoints & Spaces

03

Best Practices & Model Selection

Model selection framework, when to fine-tune vs prompt engineer

04

Hands-on Lab

Gradio demo creation and model sharing on HuggingFace Hub

Instruction Fine-Tuning: The Paradigm Shift

From Task-Specific to General Purpose

Traditional fine-tuning trains one model per task — a sentiment model, an NER model, a translation model — each requiring dedicated infrastructure and maintenance. **Instruction fine-tuning fundamentally changes this paradigm** by teaching a single model to understand and execute arbitrary natural-language instructions.



Traditional vs Instruction Fine-Tuning

TRADITIONAL APPROACH

One Model Per Task

Sentiment Model

"This is great" → **POS**

NER Model

"John lives in NY" → [**PER**,
LOC]

Translation Model

"Hello" → "**Bonjour**"

Requires a **separate model per task** — expensive to maintain, deploy, and scale.

INSTRUCTION FINE-TUNING

One Model, All Tasks

"Classify sentiment:
This is great"

"Extract entities: John
lives in NY"

"Translate to French:
Hello"

"Summarise: Long
article..."

One model handles ALL tasks — instructions replace task-specific heads, enabling zero-shot generalisation to unseen tasks.

Instruction Dataset Formats

Three dominant formats have emerged in the open-source community. Each reflects a different philosophy about how human-AI interaction should be structured. Choosing the right format for your dataset is the first decision in any instruction-tuning project.

1. Alpaca Format (Stanford)

Developed by Stanford for the original Alpaca model. Separates instruction, optional context input, and expected output into three distinct fields. Clean and simple — ideal for single-turn tasks like classification, translation, or summarisation.

```
{
  "instruction": "Classify the
sentiment of this review.",
  "input":      "This movie was
absolutely fantastic!",
  "output":     "Positive"
}
```

2. ShareGPT Format (Conversational)

Models multi-turn dialogues with alternating human/gpt turns. Originally scraped from ChatGPT conversations shared by users. Ideal for training chat models that must maintain context across multiple exchanges.

```
{
  "conversations": [
    {"from": "human", "value": "What
is the capital of France?"},
    {"from": "gpt", "value": "The
capital of France is Paris."},
    {"from": "human", "value":
"What's the population?"},
    {"from": "gpt", "value": "Paris has
a population of about 2.1 million."}
  ]
}
```

3. ChatML Format (OpenAI-style)

Uses special control tokens to delineate roles. This format is used natively by models like Mistral and Qwen, and is increasingly the de facto standard for instruction-following models requiring a system prompt.

```
<|im_start|>system
You are a helpful assistant.
<|im_end|>
<|im_start|>user
What is machine learning?
<|im_end|>
<|im_start|>assistant
Machine learning is a subset of AI
that enables
systems to learn from data.
<|im_end|>
```

Creating Instruction Datasets

From Your Data to Instruction Format

Transforming an existing labelled dataset into instruction format is straightforward. The key is writing a clear, unambiguous natural-language instruction that fully specifies what the model should do — without relying on implicit conventions.

```
# Converting classification data to instruction format
def create_instruction_dataset(examples):
    instructions = []
    for text, label in zip(examples['text'], examples['label']):
        # Create instruction-formatted example
        instruction = {
            "instruction": "Classify the sentiment of the following review as positive or negative.",
            "input":      text,
            "output":     "Positive" if label == 1 else "Negative"
        }
        instructions.append(instruction)
    return instructions

# Example transformation:
# Before: {"text": "Great product!", "label": 1}
# After: {
#     "instruction": "Classify the sentiment...",
#     "input":      "Great product!",
#     "output":     "Positive"
# }

# Formatting for model training
def format_instruction(example):
    """Format instruction for model input using Alpaca-style template."""
    if example.get("input"):
        prompt = f"""### Instruction:
{example['instruction']}

### Input:
{example['input']}

### Response:
{example['output']}"""
    else:
        prompt = f"""### Instruction:
{example['instruction']}

### Response:
{example['output']}"""
    return {"text": prompt}
```

 The `format_instruction` function handles both cases: when an auxiliary input is present (e.g. a text to classify) and when the instruction is self-contained. Always return a single `text` field to keep your dataset schema consistent across tasks.

Chat Templates

Structured Conversations for Chat Models

Different model families use incompatible chat template formats. The `apply_chat_template` method on HuggingFace tokenisers automatically handles these differences — always use it rather than hand-crafting prompt strings, which leads to subtle formatting bugs.

```
from transformers import AutoTokenizer
```

```
# ——— Llama-2 Chat Template
```

```
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")
```

```
messages = [
```

```
    {"role": "system", "content": "You are a helpful assistant."},
```

```
    {"role": "user", "content": "What is the weather like?"},
```

```
]
```

```
formatted = tokenizer.apply_chat_template(messages, tokenize=False)
```

```
# Output:
```

```
# [INST] <>
```

```
# You are a helpful assistant.
```

```
# <>
```

```
# What is the weather like? [/INST]
```

```
# ——— Mistral Chat Template
```

```
tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-Instruct-v0.1")
```

```
formatted = tokenizer.apply_chat_template(messages, tokenize=False)
```

```
# Output:
```

```
# [INST] What is the weather like? [/INST]
```

```
# Note: Mistral does not support system prompts in the same way
```

```
# ——— Phi-3 Chat Template
```

```
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")
```

```
messages = [
```

```
    {"role": "user", "content": "Hello!"},
```

```
    {"role": "assistant", "content": "Hi! How can I help?"},
```

```
    {"role": "user", "content": "What is Python?"},
```

```
]
```

```
formatted = tokenizer.apply_chat_template(messages, tokenize=False)
```

```
# Output:
```

```
# <|user|>
```

```
# Hello!<|end|>
```

```
# <|assistant|>
```

```
# Hi! How can I help?<|end|>
```

```
# <|user|>
```

```
# What is Python?<|end|>
```

```
# <|assistant|>
```


Models Suitable for Instruction Fine-Tuning

Local-Friendly Options — Choosing the Right Model for Your Hardware

The model you select should be dictated by your available VRAM and the quality bar you need to hit. Smaller models are faster to iterate on; larger models produce better outputs for complex tasks. All of the models below are compatible with QLoRA, making them trainable on consumer-grade GPUs.

Model	Parameters	Min VRAM (QLoRA)	Strengths
SmolLM2-135M	135M	2 GB	Ultra-fast experimentation; ideal for CI/CD testing
SmolLM2-360M	360M	3 GB	Good quality-for-size; fits on most laptops with a discrete GPU
SmolLM2-1.7B	1.7B	5 GB	Best small model; recommended default for instruction tuning
Phi-2	2.7B	6 GB	Strong reasoning and coding; punches above its weight class
Phi-3-mini	3.8B	8 GB	Excellent overall quality; Microsoft's flagship small model
Qwen2.5-1.5B	1.5B	4 GB	Strong multilingual support (100+ languages)
Gemma-2-2B	2B	5 GB	Google's latest; excellent instruction-following out of the box

Experimentation & Fast Iteration

SmolLM2-135M or 360M — seconds per epoch, rapid feedback loops

Quality & Practical Use

SmolLM2-1.7B, Phi-3-mini, or Gemma-2-2B — best quality/cost trade-off

Resource Constrained (4–6 GB VRAM)

SmolLM2-360M with QLoRA — achieves remarkable results in tight memory budgets

Multilingual Needs

Qwen2.5-1.5B — Alibaba's model trained on diverse multilingual corpora

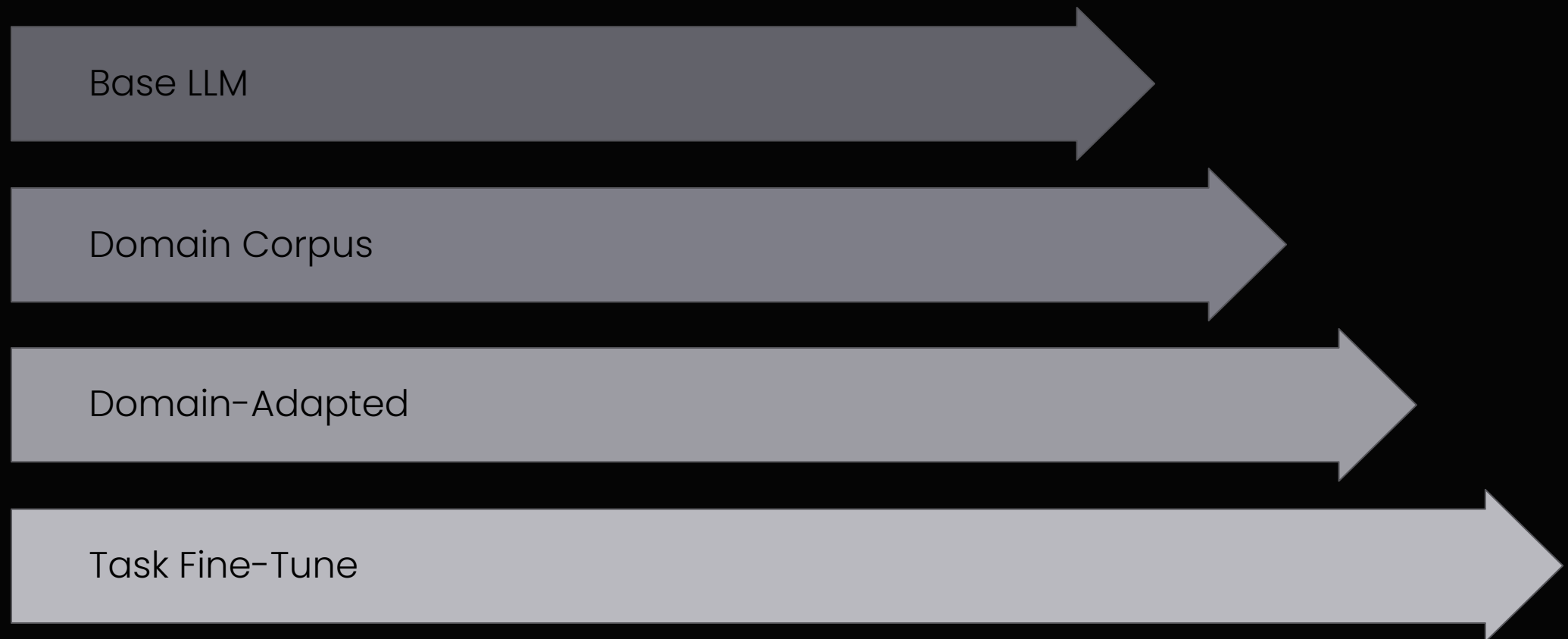


Domain Adaptation Strategies

Domain adaptation bridges the gap between a general-purpose pre-trained model and the specialised vocabulary, style, and knowledge of a target domain. There are three primary strategies, each suited to a different data regime and budget.

Strategy 1: Continued Pre-Training

Continue pre-training on a domain-specific corpus before task-specific fine-tuning. This two-stage approach lets the model internalise domain vocabulary and writing conventions before it ever sees a labelled example.



The legal domain example illustrates the pattern clearly: start with SmolLM2-1.7B, run continued pre-training on 10–100 MB of case law and statutes, then fine-tune on a small labelled set for downstream classification or summarisation. Because the model already understands legal language, far fewer labelled examples are needed in the final step.



Domain Vocabulary

Model learns specialised terminology, abbreviations, and entity names that appear rarely in general pre-training corpora.



Better Downstream Performance

Empirically, continued pre-training consistently boosts task-specific accuracy, especially when labelled data is scarce.



Less Task-Specific Data Required

Domain-adapted models generalise better from small fine-tuning sets — sometimes just dozens of examples suffice.

Strategy 2: Multi-Task Fine-Tuning

Train on multiple related tasks simultaneously by mixing them into a single instruction-formatted dataset. The model learns shared representations that transfer across tasks, and exposure to diverse objectives prevents it from forgetting how to perform any one of them.

```
# Multi-task dataset structure — all tasks share the same instruction format
```

```
dataset = {
```

```
  "legal_classification": [...], # e.g. classify contract clause type
```

```
  "legal_summarisation": [...], # e.g. summarise a case ruling
```

```
  "legal_qa":           [...], # e.g. answer questions about a statute
```

```
  "legal_ner":          [...], # e.g. extract parties, dates, courts
```

```
}
```

Prevents Catastrophic Forgetting

Mixing tasks means the model is never exclusively optimised for one objective, preserving performance across all tasks throughout training.

Improves Generalisation

Exposure to diverse task formats teaches the model to follow instructions more robustly, including on held-out tasks it was never explicitly trained on.

Shared Representations

Related tasks share useful intermediate representations — e.g. NER and QA both benefit from precise entity grounding — leading to mutual improvement.

Strategy 3: Data Augmentation Techniques

When labelled data is limited, augmentation can multiply your dataset size without additional annotation cost. The following techniques are especially effective for NLP instruction datasets.

```
import random
from nltk.corpus import wordnet

# — 1. Back Translation

def back_translate(text, source_lang="en", target_lang="fr"):
    """Translate to French and back to English — produces paraphrases."""
    fr_text = translator(text, source_lang, target_lang)
    back_text = translator(fr_text, target_lang, source_lang)
    return back_text # Similar meaning, different surface form
```

```
# — 2. Synonym Replacement
```

```
def synonym_replacement(text, n=2):
    """Randomly replace n words with WordNet synonyms."""
    words = text.split()
    for _ in range(n):
        idx = random.randint(0, len(words) - 1)
        synonyms = wordnet.synsets(words[idx])
        if synonyms:
            synonym = random.choice(synonyms).lemmas()[0].name()
            words[idx] = synonym
    return ' '.join(words)
```

```
# — 3. Prompt Variation
```

```
# Write multiple phrasings of the same instruction to improve robustness
```

```
prompts = [
    "Classify the sentiment: {text}",
    "What is the sentiment of this review? {text}",
    "Determine if this is positive or negative: {text}",
    "Sentiment analysis: {text}",
    "Is this review positive or negative? {text}",
]
```

```
# — 4. Easy Data Augmentation (EDA)
```

```
# Random insertion: insert a random synonym of a random word at a random position
# Random deletion: randomly remove each word with probability p
# Random swap: randomly swap two words in the sentence n times
# Synonym insertion: insert a synonym of a non-stop word at a random location
```

📌 Prompt variation is the most impactful augmentation for instruction datasets. A model that sees five different phrasings of "classify the sentiment" is far more robust to novel instruction wording at inference time.



Introducing Unsloth

Ultra-Fast Fine-Tuning Framework

Unsloth is an open-source fine-tuning framework that achieves **2–5× faster training** and **50–80% lower memory usage** compared to standard HuggingFace + PEFT implementations — without changing your training results. It achieves this by replacing generic PyTorch operations with model-specific, hand-optimised kernels.

Unsloth: Key Features at a Glance



2–5× Faster Training

Custom CUDA kernels and Triton optimisations replace generic PyTorch operations with model-specific implementations that exploit known architectural patterns.



50–80% Less Memory

Manual autograd overrides, fused cross-entropy loss, and chunked attention dramatically reduce peak GPU memory — enabling larger batches or longer sequences on the same hardware.



Open-Source, MIT Licensed

Fully open, no proprietary runtime required. Contributions are welcome and the community actively improves kernel coverage for new model families.



Native HuggingFace Integration

Drop-in replacement for standard Transformers + PEFT. Use the same `Trainer` API, same config files, same checkpoints — just faster.



Direct GGUF Export

Export trained models directly to GGUF format for use with llama.cpp, Ollama, and LM Studio — no intermediate conversion step required.



Broad Model Coverage

Optimised kernels for Llama, Mistral, Phi, Gemma, Qwen, SmolLM2, DeepSeek, and vision-language models including Llama-3.2-Vision and Qwen2-VL.

How Unsloth Achieves Speedups

Three Core Technical Innovations

Unsloth's performance gains stem from three targeted engineering decisions, each addressing a different bottleneck in the standard training loop. Together they compound to deliver the headline 2–5× improvement.

1. Manual Autograd Overrides

Standard PyTorch's autograd engine traces every operation in the forward pass to construct a computation graph for the backward pass. This is flexible but memory-heavy. Unsloth replaces the attention and MLP backward passes with manually written gradient functions that are algebraically equivalent but allocate 30–50% less memory. No intermediate activations need to be stored for re-materialisation during the backward pass.

2. Optimised Cross-Entropy Loss

Standard implementations materialise the full logit matrix of shape `(batch_size × seq_len × vocab_size)` before computing the loss. For a 32K-vocabulary model with a batch of 4 sequences of length 2048, this is a $4 \times 2048 \times 32000 = 262\text{M}$ float tensor — roughly 1 GB of VRAM just for logits. Unsloth's fused kernel computes the cross-entropy loss in a single pass without ever materialising this matrix, saving 2–4 GB per training step.

3. Chunked (Block) Attention

Standard self-attention requires $O(n^2)$ memory with respect to sequence length because the full $n \times n$ attention matrix must be held in memory simultaneously. Unsloth processes attention in fixed-size chunks, similar in spirit to Flash Attention, reducing memory complexity for long sequences by approximately 50%. This makes training on sequences of 4096+ tokens feasible on 8–12 GB GPUs.

Unsloth Performance Benchmarks

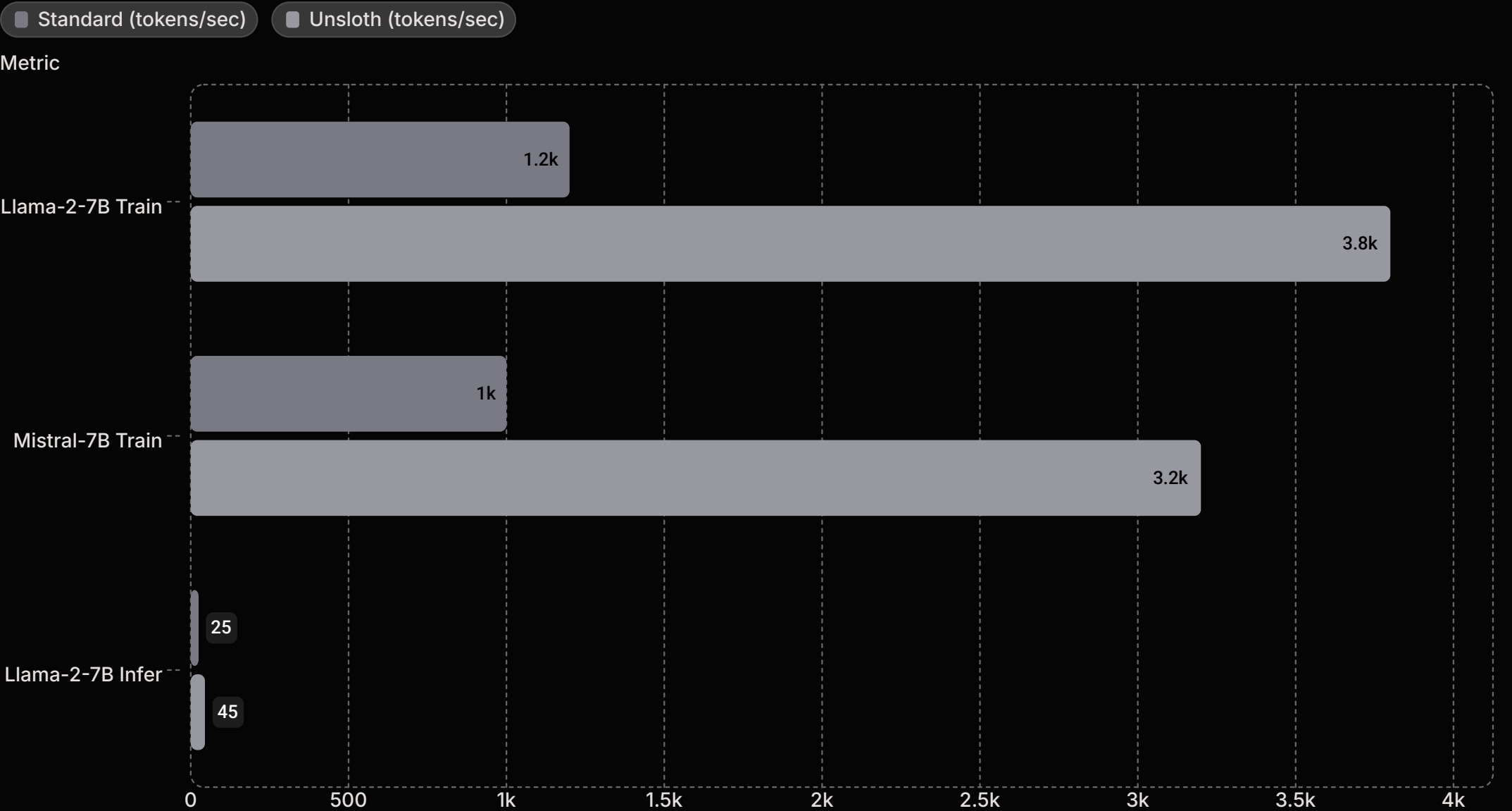
Real-World Speed and Memory Comparisons on a Single GPU

Llama-2-7B — QLoRA, Single GPU

Metric	Standard	Unsloth	Speedup
Training Time/Step	2.1 s	0.7 s	3.0×
Peak GPU Memory	15.2 GB	9.8 GB	1.6×
Tokens/sec (train)	1,200	3,800	3.2×
Tokens/sec (infer)	25	45	1.8×

Mistral-7B — QLoRA, Single GPU

Metric	Standard	Unsloth	Speedup
Training Time/Step	2.5 s	0.8 s	3.1×
Peak GPU Memory	16.8 GB	10.2 GB	1.6×
Tokens/sec (train)	1,000	3,200	3.2×



Unsloth consistently delivers **2–5× speedup** and **50–80% memory reduction** — without any change to model accuracy or training hyperparameters.

Unsloth: Supported Models

Model Compatibility Matrix (as of 2025)

Llama Family

- Llama-3.2 (1B, 3B)
- Llama-3.1 (8B)
- Llama-3 (8B, 70B)
- Llama-2 (7B, 13B, 70B)

Mistral Family

- Mistral 7B
- Mixtral 8×7B
- Mixtral 8×22B

Phi Family

- Phi-3 (mini, small, medium)
- Phi-3.5
- Phi-4

Gemma Family

- Gemma-2 (2B, 9B, 27B)

Qwen Family

- Qwen2.5 (0.5B to 72B)

Others & Vision

- SmolLM2, TinyLlama, Zephyr
- CodeLlama, DeepSeek-R1-Distill
- Llama-3.2-Vision, Pixtral, Qwen2-VL

Unsloth vs Standard Fine-Tuning

Side-by-Side Code Comparison

The Unsloth API is designed to be a near-drop-in replacement. The critical difference is the `FastLanguageModel` wrapper, which patches the model internals with optimised kernels at load time.

STANDARD APPROACH

```
from transformers import (
    AutoModelForCausalLM,
    Trainer
)
from peft import LoraConfig, get_peft_model

# Load base model with 4-bit quantisation
model = AutoModelForCausalLM.from_pretrained(
    "unsloth/Llama-3.2-3B",
    load_in_4bit=True,
)

# Attach LoRA adapters via PEFT
lora_config = LoraConfig(
    r=16,
    lora_alpha=16,
    target_modules=[
        "q_proj", "k_proj", "v_proj",
        "o_proj", "gate_proj",
        "up_proj", "down_proj"
    ],
)
model = get_peft_model(model, lora_config)

# Standard Trainer
trainer = Trainer(model=model, ...)
trainer.train()

# Training time: ~2 hours for 1K examples
```

UNSLOTH APPROACH

```
from unsloth import FastLanguageModel

# Load model — kernels patched automatically
model, tokenizer = FastLanguageModel.from_pretrained(
    "unsloth/Llama-3.2-3B",
    load_in_4bit=True,
)

# Attach LoRA adapters via Unsloth API
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj", "k_proj", "v_proj",
        "o_proj", "gate_proj",
        "up_proj", "down_proj"
    ],
)

# Same Trainer API — fully compatible
trainer = Trainer(model=model, ...)
trainer.train()

# Training time: ~45 min (2.7× faster!)
```

Unsloth Installation and Setup

```
# ——— Installation
```

```
# pip install unsloth
# pip install --upgrade unsloth # For the latest stable release
# pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
```

```
import torch
from unsloth import FastLanguageModel
```

```
# ——— Step 1: Load model with Unsloth
```

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Llama-3.2-3B", # Any HuggingFace model ID works
    max_seq_length = 2048, # Reduce for memory savings on smaller GPUs
    dtype = None, # Auto-detect: float16 on Ampere, bfloat16 on Hopper
    load_in_4bit = True, # 4-bit NF4 quantisation (QLoRA)
    # load_in_8bit = False, # Alternative: 8-bit quantisation (less compression)
    # full_finetuning = False, # Alternative: full fine-tuning (requires more VRAM)
)
```

```
# ——— Step 2: Attach LoRA adapters
```

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # LoRA rank — higher = more capacity, more VRAM
    target_modules = [ # Which linear projections to adapt
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha = 16, # Scaling factor (keep equal to r as a baseline)
    lora_dropout = 0, # 0 is optimal for Unsloth — no extra regularisation
    bias = "none",
    use_gradient_checkpointing = "unsloth", # Unsloth's optimised GC (30% less than standard)
    random_state = 42,
    use_rslora = False, # Rank-stabilised LoRA — useful for higher ranks
    loftq_config = None, # LoftQ initialisation — alternative to standard random init
)
```

Unsloth Key Features — Part 1

FastLanguageModel Wrapper and Optimised Gradient Checkpointing

```
# — Feature 1: FastLanguageModel Wrapper —  
# The wrapper patches the loaded model with Unsloth's custom kernels at load time.  
# You can switch between training and inference modes explicitly.
```

```
model, tokenizer = FastLanguageModel.from_pretrained(  
    "meta-llama/Llama-3.2-3B",  
    load_in_4bit=True,  
)  
# Model is automatically patched — no further action needed.
```

```
# Switch to 2× faster inference mode (disables dropout, fuses layers)  
FastLanguageModel.for_inference(model)
```

```
# Switch back to 2–5× faster training mode  
FastLanguageModel.for_training(model)
```

```
# — Feature 2: Unsloth's Optimised Gradient Checkpointing —  
# Gradient checkpointing trades computation for memory by re-computing activations  
# during the backward pass instead of storing them.  
# Unsloth's implementation uses 30% less memory than PyTorch's standard version.
```

```
model = FastLanguageModel.get_peft_model(  
    model,  
    r=16,  
    use_gradient_checkpointing="unsloth", # Recommended: Unsloth's optimised version  
    # use_gradient_checkpointing=True, # Standard PyTorch implementation  
    # use_gradient_checkpointing=False, # Disabled: fastest forward, most memory  
)
```

Unsloth Key Features — Part 2

Direct GGUF Export and Continued Pre-Training Support

```
# — Feature 3: Direct GGUF Export —  
# Standard workflow: model → HuggingFace → llama.cpp conversion → GGUF  
# Unsloth workflow: model → GGUF (one step, no intermediate conversion)
```

```
model.save_pretrained_gguf(  
    "my-model-gguf",  
    tokenizer,  
    quantization_method=[  
        "q4_k_m", # 4-bit, medium quality — recommended for most use cases  
        "q5_k_m", # 5-bit, medium quality — slightly better than q4 at cost of size  
        "q8_0", # 8-bit — near-lossless quality, largest file  
        "f16", # 16-bit float — no quantisation, reference quality  
    ]  
)  
# Creates: my-model-gguf/  
# |—— model-q4_k_m.gguf ← Ready for Ollama, LM Studio, llama.cpp  
# |—— model-q5_k_m.gguf  
# |—— model-q8_0.gguf  
# |—— model-f16.gguf
```

```
# — Feature 4: Continued Pre-Training Support —  
# Unsloth supports continued pre-training (unsupervised next-token prediction)  
# in addition to supervised instruction fine-tuning.
```

```
model = FastLanguageModel.get_peft_model(  
    model,  
    r=16,  
    use_gradient_checkpointing="unsloth",  
)  
  
# Use the standard Trainer with a language modelling collator  
trainer = Trainer(  
    model = model,  
    args = training_args,  
    train_dataset = domain_corpus, # Raw domain text — no labels needed  
    data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False),  
)
```


Lab: Instruction Fine-Tuning with Unsloth

45-Minute End-to-End Walkthrough

01

Environment and Data Preparation

Install Unsloth, construct a multi-task instruction dataset covering sentiment analysis, summarisation, QA, code generation, and general knowledge.

02

Load Model with Unsloth

Load SmolLM2-360M (or larger) with 4-bit quantisation and configure LoRA adapters via `FastLanguageModel.get_peft_model`.

03

Tokenise and Configure Training

Tokenise instruction-formatted prompts, configure `TrainingArguments` with gradient accumulation and mixed precision.

04

Train and Evaluate

Run `trainer.train()`, monitor loss curves, evaluate on held-out test set.

05

Test and Export

Test instruction-following capabilities on seen and unseen tasks, then export to GGUF and merged 16-bit formats.

Lab Exercise 1: Environment and Data Preparation

```
print("=" * 60)
print("EXERCISE 1: Setup and Data Preparation")
print("=" * 60)

# Install Unsloth if not already available
# !pip install unsloth datasets

import torch
from datasets import Dataset, load_dataset
from transformers import TrainingArguments, Trainer, DataCollatorForSeq2Seq
from unsloth import FastLanguageModel, is_bfloat16_supported

# ——— Check environment ———

print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memory: {torch.cuda.get_device_properties(0).total_mem / 1e9:.1f} GB")

# ——— Build a multi-task instruction dataset ———
instruction_data = [
    # Task 1: Sentiment Analysis
    {
        "instruction": "Classify the sentiment of the following text as positive, negative, or neutral.",
        "input": "I absolutely loved this movie! The acting was superb.",
        "output": "Positive"
    },
    {
        "instruction": "Classify the sentiment of the following text as positive, negative, or neutral.",
        "input": "This product broke after two days. Very disappointed.",
        "output": "Negative"
    },
    {
        "instruction": "Classify the sentiment of the following text as positive, negative, or neutral.",
        "input": "The package arrived on time. It was as described.",
        "output": "Neutral"
    },
    # Task 2: Summarisation
    {
        "instruction": "Summarise the following text in one sentence.",
        "input": "The Industrial Revolution, which began in Britain in the late 18th century, "
        "marked a major turning point in human history. It transformed largely agrarian, "
        "rural societies into industrialised, urban ones.",
        "output": "The Industrial Revolution transformed agrarian societies into industrialised urban centres."
    },
    {
        "instruction": "Summarise the following text in one sentence.",
        "input": "Photosynthesis is the process by which green plants and some other organisms use "
        "sunlight to synthesise foods with the help of chlorophyll. It is essential for "
        "life on Earth as it produces oxygen.",
        "output": "Photosynthesis is the process by which plants convert sunlight into food and produce oxygen."
    },
    # Task 3: Question Answering
    {
        "instruction": "Answer the question based on the provided information.",
        "input": "Information: The Great Wall of China is over 13,000 miles long and was built over "
        "many centuries. Question: How long is the Great Wall of China?",
        "output": "The Great Wall of China is over 13,000 miles long."
    },
    {
        "instruction": "Answer the question based on the provided information.",
        "input": "Information: Water boils at 100 degrees Celsius at sea level. At higher altitudes, "
        "the boiling point decreases. Question: At what temperature does water boil at sea level?",
        "output": "Water boils at 100 degrees Celsius at sea level."
    },
    # Task 4: Code Generation
    {
        "instruction": "Write a simple function to solve the given problem.",
        "input": "Problem: Write a Python function that returns the factorial of a number.",
        "output": "def factorial(n):\n if n == 0:\n return 1\n return n * factorial(n - 1)"
    },
    {
        "instruction": "Write a simple function to solve the given problem.",
        "input": "Problem: Write a Python function that checks if a string is a palindrome.",
        "output": "def is_palindrome(s):\n s = s.lower()\n return s == s[::-1]"
    },
    # Task 5: General Knowledge
    {
        "instruction": "Provide a brief answer to the following question.",
        "input": "Question: What is machine learning?",
        "output": "Machine learning is a subset of artificial intelligence that enables systems to "
        "learn and improve from experience without being explicitly programmed."
    },
]

# ——— Augment with prompt variations for the sentiment task ———
varied_data = []
prompt_templates = [
    "Classify the sentiment: {input}",
    "What is the sentiment of this? {input}",
    "Is this positive or negative? {input}",
    "Determine the sentiment: {input}",
]

for item in instruction_data:
    if "sentiment" in item["instruction"].lower():
        for template in prompt_templates:
            varied_data.append({
                "instruction": template.format(input=item["input"]),
                "input": "",
                "output": item["output"]
            })
    else:
        varied_data.append(item)

all_data = instruction_data + varied_data
print(f"\nTotal training examples: {len(all_data)}")

# ——— Format as instruction prompts ———
def format_instruction(example):
    """Format a single example as an Alpaca-style instruction prompt."""
    if example.get("input", "").strip():
        text = f"""### Instruction:
{example['instruction']}

### Input:
{example['input']}

### Response:
{example['output']}"""
    else:
        text = f"""### Instruction:
{example['instruction']}

### Response:
{example['output']}"""
    return {"text": text}

# ——— Create HuggingFace Dataset ———
dataset = Dataset.from_list(all_data)
dataset = dataset.map(format_instruction)
dataset = dataset.train_test_split(test_size=0.15, seed=42)

print(f"Train size: {len(dataset['train'])}")
print(f"Test size: {len(dataset['test'])}")
print(f"\nSample formatted text:\n{dataset['train'][0]['text'][:300]}...")
```

Lab Exercise 2: Load Model with Unsloth

```
print("\n" + "=" * 60)
print("EXERCISE 2: Load Model with Unsloth")
print("=" * 60)

from unsloth import FastLanguageModel
import torch

# ——— Choose model based on available GPU memory ———
# For 4–6 GB GPUs: "HuggingFaceTB/SmolLM2-360M"
# For 6–8 GB GPUs: "HuggingFaceTB/SmolLM2-1.7B"
# For 8+ GB GPUs: "microsoft/Phi-3-mini-4k-instruct"

model_name = "HuggingFaceTB/SmolLM2-360M" # Safe default for most hardware
max_seq_length = 1024 # Reduce if you hit OOM errors

# ——— Load with Unsloth optimisations ———
print(f"Loading {model_name} with Unsloth...")
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = model_name,
    max_seq_length = max_seq_length,
    dtype = None, # Auto-detect best dtype for your GPU
    load_in_4bit = True, # 4-bit NF4 quantisation
)

print(f"\nModel loaded successfully!")
print(f"Model type: {type(model).__name__}")
print(f"Tokenizer vocab size:{tokenizer.vocab_size}")

# Ensure pad token is set (required for batched training)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# ——— Configure LoRA adapters ———

model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # LoRA rank
    target_modules = [ # All attention + FFN projection layers
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha = 16,
    lora_dropout = 0.05, # Small dropout for regularisation
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
    use_rslora = False,
)

# ——— Parameter analysis ———

trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total = sum(p.numel() for p in model.parameters())
print(f"\nParameter Analysis:")
print(f" Trainable: {trainable:,} ({100 * trainable / total:.2f}%)")
print(f" Total: {total:,}")
print(f" Efficiency:{total / trainable:.0f}× fewer trainable params vs full fine-tuning")

if torch.cuda.is_available():
    memory = torch.cuda.memory_allocated() / 1e9
    print(f" GPU Memory used: {memory:.2f} GB")
```

Lab Exercise 3: Tokenisation and Training Setup

```
print("\n" + "=" * 60)
print("EXERCISE 3: Tokenisation and Training Setup")
print("=" * 60)

# ——— Tokenisation function


---



def tokenize_function(examples):
    """Tokenise instruction texts for causal language modelling."""
    model_inputs = tokenizer(
        examples["text"],
        truncation = True,
        padding = "max_length",
        max_length = max_seq_length,
    )
    # For causal LM, labels are identical to input_ids.
    # The loss is computed only on non-padding tokens.
    model_inputs["labels"] = model_inputs["input_ids"].copy()
    return model_inputs

# ——— Tokenise datasets


---



print("Tokenising datasets...")
tokenized_train = dataset['train'].map(
    tokenize_function,
    batched = True,
    remove_columns = dataset['train'].column_names,
)
tokenized_test = dataset['test'].map(
    tokenize_function,
    batched = True,
    remove_columns = dataset['test'].column_names,
)
print(f"Tokenised train size: {len(tokenized_train)}")
print(f"Tokenised test size: {len(tokenized_test)}")

# ——— TrainingArguments


---



from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir = "./unsloth-instruction-model",

    # Training duration
    num_train_epochs = 3,
    per_device_train_batch_size = 4,
    per_device_eval_batch_size = 8,
    gradient_accumulation_steps = 4, # Effective batch = 4 × 4 = 16

    # Optimisation
    learning_rate = 2e-4, # Higher LR is fine for LoRA
    warmup_steps = 25,
    weight_decay = 0.01,
    optim = "adamw_8bit", # Saves memory vs standard AdamW

    # Logging and saving
    logging_steps = 10,
    logging_first_step = True,
    evaluation_strategy = "steps",
    eval_steps = 50,
    save_strategy = "steps",
    save_steps = 50,
    save_total_limit = 2,
    load_best_model_at_end = True,
    metric_for_best_model = "eval_loss",
    greater_is_better = False,

    # Precision
    fp16 = not is_bfloat16_supported(),
    bf16 = is_bfloat16_supported(),
    gradient_checkpointing = True,

    report_to = "none",
    seed = 42,
)

print(f"\nTraining Configuration:")
print(f" Epochs: {training_args.num_train_epochs}")
print(f" Batch size (device): {training_args.per_device_train_batch_size}")
print(f" Gradient accumulation:{training_args.gradient_accumulation_steps}")
print(f" Effective batch: {training_args.per_device_train_batch_size * training_args.gradient_accumulation_steps}")
print(f" Learning rate: {training_args.learning_rate}")
print(f" FP16: {training_args.fp16} | BF16: {training_args.bf16}")
```


Lab Exercise 4: Training the Model

```
print("\n" + "=" * 60)
print("EXERCISE 4: Training the Model")
print("=" * 60)

from transformers import Trainer
from unsloth import is_bfloat16_supported
import time

# ——— Instantiate Trainer


---



trainer = Trainer(
    model      = model,
    args       = training_args,
    train_dataset = tokenized_train,
    eval_dataset = tokenized_test,
    tokenizer   = tokenizer,
)

# ——— Train


---



print("\nStarting instruction fine-tuning...")
print("=" * 50)

start_time  = time.time()
trainer.train()
training_time = time.time() - start_time

print(f"\nTraining completed in {training_time / 60:.1f} minutes")

# ——— Evaluate on held-out test set


---



eval_results = trainer.evaluate()
print(f"\nEvaluation Results:")
for key, value in eval_results.items():
    if isinstance(value, float):
        print(f" {key}: {value:.4f}")

# ——— Save model and tokeniser


---



print(f"\nSaving fine-tuned model...")
model.save_pretrained("./fine-tuned-instruction-model")
tokenizer.save_pretrained("./fine-tuned-instruction-model")
print("Model saved!")
```

 Monitor the gap between **train_loss** and **eval_loss** during training. A widening gap indicates overfitting — reduce epochs or lower the LoRA rank if this occurs.

Lab Exercise 5: Testing Instruction Following

```
print("\n" + "=" * 60)
print("EXERCISE 5: Testing the Model")
print("=" * 60)

# —— Switch to inference mode (2× faster) ——
FastLanguageModel.for_inference(model)

# —— Diverse test prompts

test_prompts = [
    # Seen task variant: sentiment
    "### Instruction:\nClassify the sentiment of the following text.\n\n"
    "### Input:\nThis restaurant is amazing! Best food ever.\n\n### Response:\n",

    # Seen task variant: summarisation
    "### Instruction:\nSummarise the following text in one sentence.\n\n"
    "### Input:\nArtificial intelligence has revolutionised many industries including healthcare, "
    "finance, and transportation. Machine learning algorithms can now detect diseases, predict "
    "market trends, and enable autonomous driving.\n\n### Response:\n",

    # Seen task variant: QA
    "### Instruction:\nAnswer the question based on the provided information.\n\n"
    "### Input:\nInformation: The Python programming language was created by Guido van Rossum "
    "and first released in 1991. Question: Who created Python?\n\n### Response:\n",

    # Unseen but related: style transfer
    "### Instruction:\nRewrite the following sentence to be more formal.\n\n"
    "### Input:\nHey, can you tell me what time the meeting starts?\n\n### Response:\n",

    # Creative: unseen task
    "### Instruction:\nWrite a short poem about technology.\n\n### Response:\n",
]

print("Testing Instruction-Following Capabilities:")
print("=" * 50)

for i, prompt in enumerate(test_prompts, 1):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens = 100,
            do_sample = True,
            temperature = 0.7,
            top_p = 0.9,
            top_k = 50,
            pad_token_id = tokenizer.pad_token_id,
            eos_token_id = tokenizer.eos_token_id,
        )

    full_response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    generated_part = full_response[len(prompt):].strip()

    # Extract first complete response block
    if '\n' in generated_part:
        generated_part = generated_part.split("\n")[0]

    print(f"\nTest {i}:")
    print(f" Prompt: {prompt.split('### Response:')[0][:100]}...")
    print(f" Response: {generated_part[:200]}")
    print("-" * 50)
```

Lab Exercise 6: Export and Advanced Features

```
print("\n" + "=" * 60)
print("EXERCISE 6: Export and Advanced Features")
print("=" * 60)
```

```
import os
```

```
# —— Export to GGUF (for local inference with Ollama/llama.cpp) ——
```

```
print("Attempting GGUF export...")
```

```
try:
```

```
    model.save_pretrained_gguf(
```

```
        "./instruction-model-gguf",
```

```
        tokenizer,
```

```
        quantization_method=["q4_k_m", "q8_0"]
```

```
    )
```

```
    print("
```


Key Takeaways — Module 9



Instruction Fine-Tuning

Enables a single model to follow diverse natural-language instructions. Key formats include Alpaca (single-turn), ShareGPT (multi-turn conversational), and ChatML (OpenAI-style with control tokens). Always use `apply_chat_template` for model-specific formatting.



Domain Adaptation

Three complementary strategies: continued pre-training on domain text, multi-task fine-tuning to prevent catastrophic forgetting, and data augmentation (back translation, synonym replacement, prompt variation) to expand small datasets.



Unsloth Framework

Delivers 2–5× training speedup and 50–80% memory reduction via manual autograd overrides, fused cross-entropy loss, and chunked attention. Drop-in replacement for standard PEFT with native HuggingFace `Trainer` compatibility.



Practical Skills Acquired

Creating instruction datasets, converting raw labels to instruction format, training with Unsloth on consumer GPUs, testing instruction-following generalisation, and exporting to GGUF for local deployment.



Next up — **Module 10**: Model evaluation and benchmarking, deployment with Gradio, and model sharing best practices on HuggingFace Hub.

Additional Resources — Module 9

Key Papers

- [Training Language Models to Follow Instructions \(InstructGPT\)](#)
- [Self-Instruct: Aligning Language Models with Self-Generated Instructions](#)

Tools and Documentation

- [Unsloth GitHub Repository](#)
- [Unsloth Documentation](#)
- [HuggingFace Instruction Fine-Tuning Guide](#)
- [HuggingFace Chat Templates Reference](#)

Quick Reference: LoRA Hyperparameter Guide

Rank r

Start with 16. Increase to 32–64 for complex tasks; decrease to 8 for tiny models or very constrained memory.

Alpha

Keep equal to r as a safe default. Higher alpha relative to rank increases the effective learning rate of adapters.

Learning Rate

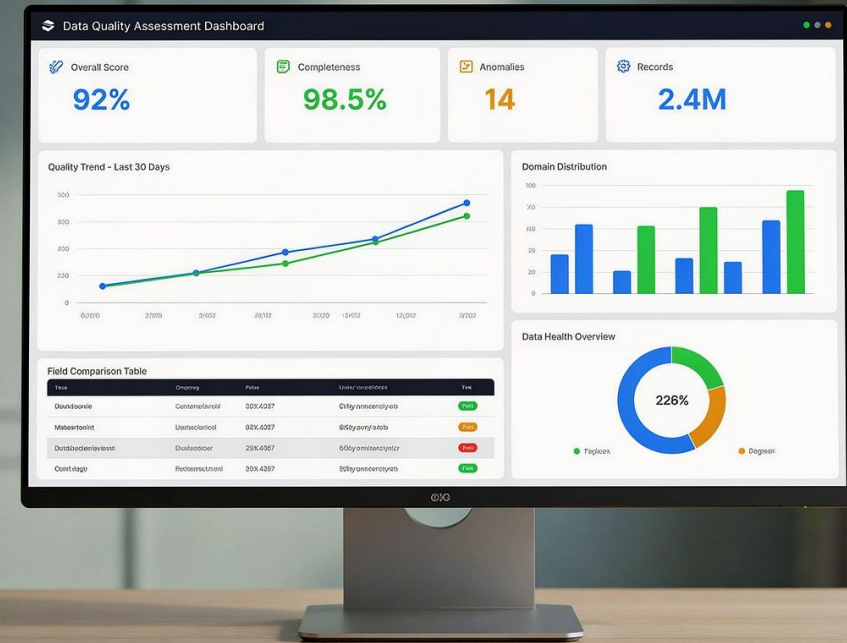
$2e-4$ is a good default for LoRA. LoRA can tolerate higher LRs than full fine-tuning because only a small fraction of weights are updated.

MODULE 10

Deployment, Evaluation & Best Practices

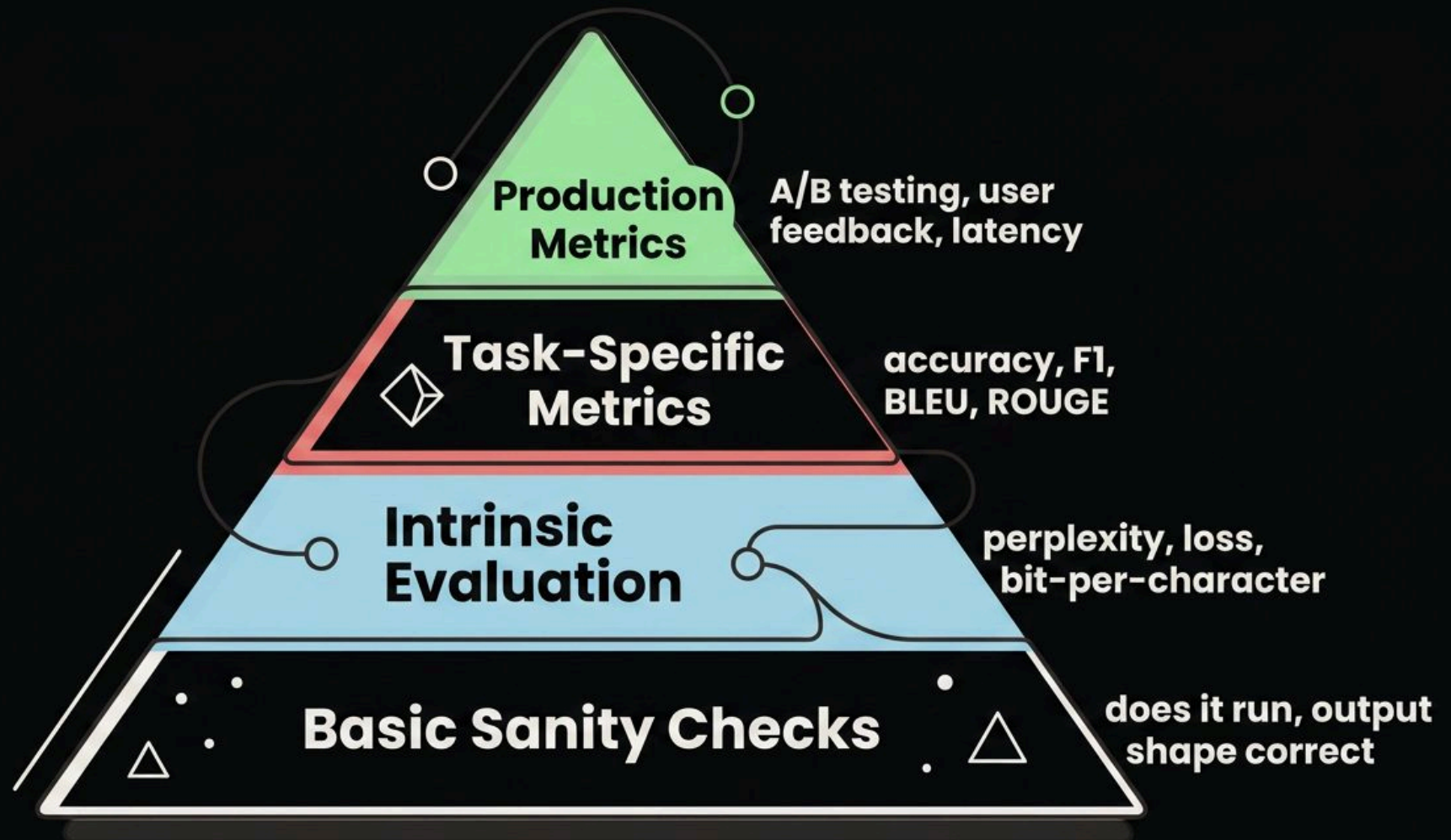
From Model to Production

A well-trained model is only as valuable as your ability to measure its quality, deploy it reliably, and maintain it in production. Module 10 closes the loop on the full fine-tuning lifecycle.



Why Evaluate? The Evaluation Pyramid

Rigorous evaluation is the difference between a model that works in a notebook and one that performs reliably in production. Each layer of the evaluation pyramid validates a different aspect of model quality — skipping layers leads to blind spots that only appear when it is too late.



Each level validates a different aspect of model quality.

Begin with sanity checks before investing compute in full evaluation runs. Work up the pyramid systematically — a model with poor intrinsic metrics will almost never achieve strong task-specific scores, and task-specific scores are a necessary but not sufficient condition for production success.

Intrinsic vs Extrinsic Evaluation

INTRINSIC

Measuring Internal Quality

Intrinsic metrics measure how well the model has learned the statistical structure of language, independently of any downstream task. They are fast to compute and useful for comparing training runs or detecting regressions during continued pre-training.

Perplexity (PPL)

How "surprised" is the model by held-out text? Lower is better. $PPL = \exp(\text{cross_entropy_loss})$. A drop from 15.2 to 12.8 indicates meaningful improvement.

Cross-Entropy Loss

The direct training objective. Tracks on both train and validation sets to detect overfitting.

Bits-Per-Character (BPC)

Information-theoretic measure of compression quality. Useful for comparing models across different tokenisations.

EXTRINSIC

Measuring Task Performance

Extrinsic metrics measure performance on the actual downstream task. These are what end users and stakeholders care about. Always define your primary extrinsic metric *before* starting training to avoid metric shopping.

Accuracy / F1

Classification tasks. F1 is preferred over accuracy for imbalanced class distributions.

BLEU / METEOR

Translation quality. METEOR is more robust than BLEU as it accounts for synonyms and morphological variants.

ROUGE-1/2/L

Summarisation quality. Measures n-gram and longest-common-subsequence overlap between generated and reference summaries.

BERTScore / Human Eval

Semantic similarity using contextual embeddings. Human evaluation remains the gold standard for generative tasks.

Using the `evaluate` Library

Standard Metrics Made Easy

```
from evaluate import load
import numpy as np
```

——— 1. Accuracy

```
accuracy = load("accuracy")
results = accuracy.compute(
    predictions=[0, 1, 1, 0],
    references=[0, 1, 0, 0]
)
print(f"Accuracy: {results['accuracy']:.3f}") # 0.750
```

——— 2. F1 Score

```
f1 = load("f1")
results = f1.compute(
    predictions=[0, 1, 1, 0],
    references=[0, 1, 0, 0],
    average="weighted"
)
print(f"F1: {results['f1']:.3f}")
```

——— 3. BLEU (translation / open-ended generation)

```
bleu = load("bleu")
results = bleu.compute(
    predictions=["The cat is on the mat"],
    references=[["The cat sat on the mat"]]
)
print(f"BLEU: {results['bleu']:.3f}")
```

——— 4. ROUGE (summarisation)

```
# ROUGE = Recall-Oriented Understudy for Gisting Evaluation
rouge = load("rouge")
results = rouge.compute(
    predictions=["The cat sat on the mat"],
    references=["A cat was sitting on the mat"]
)
print(f"ROUGE-1: {results['rouge1']:.3f}") # Unigram overlap
print(f"ROUGE-2: {results['rouge2']:.3f}") # Bigram overlap
print(f"ROUGE-L: {results['rougeL']:.3f}") # Longest common subsequence
```

——— 5. Perplexity

```
perplexity = load("perplexity")
results = perplexity.compute(
    predictions=["The cat sat on the mat."],
    model_id="gpt2"
)
print(f"Perplexity: {results['mean_perplexity']:.2f}")
```

Comprehensive Evaluation Function

```
def evaluate_model_comprehensive(model, tokenizer, test_dataset, task_type):
    """
    Runs a full evaluation suite for classification or generation tasks.

    Parameters
    -----
    model : fine-tuned HuggingFace model
    tokenizer : corresponding tokeniser
    test_dataset : DataLoader or HF Dataset
    task_type : "classification" | "generation"

    Returns
    -----
    results : dict of metric name → value
    all_preds : list of predictions (classification) or generated strings (generation)
    all_labels : list of ground-truth labels (classification only)
    """

    from evaluate import load
    import numpy as np
    from tqdm.auto import tqdm

    results = {}

    if task_type == "classification":
        accuracy = load("accuracy")
        f1 = load("f1")
        precision = load("precision")
        recall = load("recall")

        all_preds, all_labels = [], []
        model.eval()

        for batch in tqdm(test_dataset, desc="Evaluating"):
            with torch.no_grad():
                outputs = model(**batch)
                preds = torch.argmax(outputs.logits, dim=-1)
                all_preds.extend(preds.cpu().numpy())
                all_labels.extend(batch['labels'].cpu().numpy())

        results["accuracy"] = accuracy.compute(
            predictions=all_preds, references=all_labels)["accuracy"]
        results["f1"] = f1.compute(
            predictions=all_preds, references=all_labels, average="weighted")["f1"]
        results["precision"] = precision.compute(
            predictions=all_preds, references=all_labels, average="weighted")["precision"]
        results["recall"] = recall.compute(
            predictions=all_preds, references=all_labels, average="weighted")["recall"]

    elif task_type == "generation":
        rouge = load("rouge")
        bleu = load("bleu")

        all_preds, all_refs = [], []
        model.eval()

        for batch in tqdm(test_dataset, desc="Evaluating"):
            with torch.no_grad():
                outputs = model.generate(**batch['input_ids'])
                preds = tokenizer.batch_decode(outputs, skip_special_tokens=True)
                refs = batch['references']
                all_preds.extend(preds)
                all_refs.extend(refs)

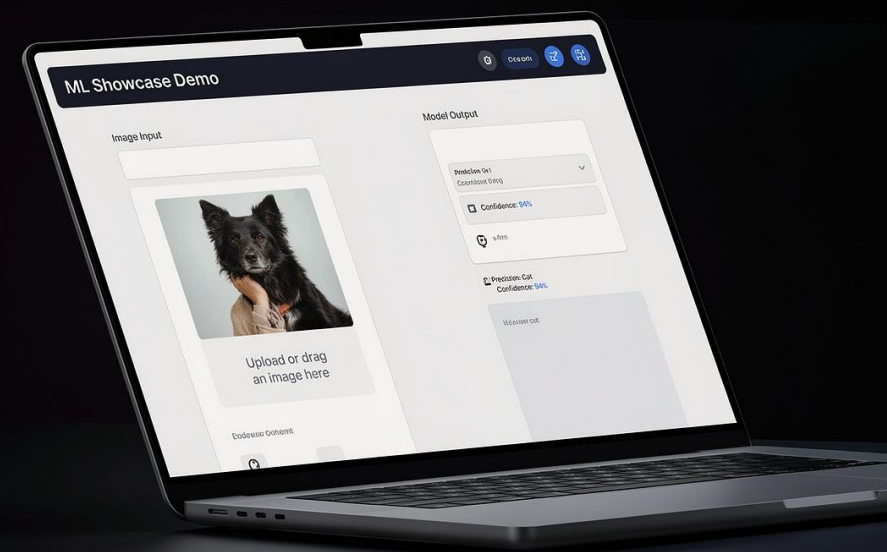
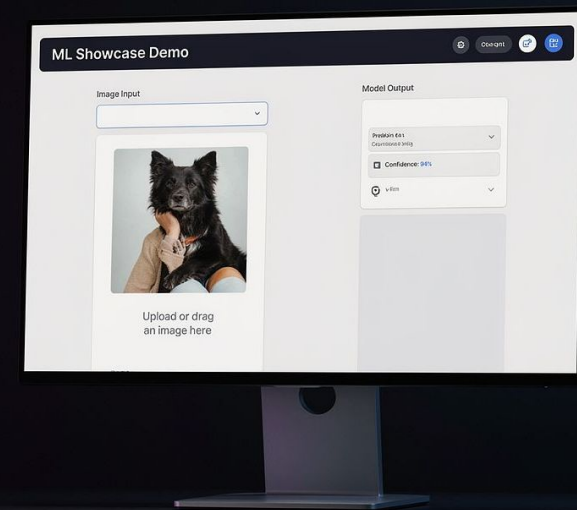
        results["rouge"] = rouge.compute(
            predictions=all_preds, references=all_refs)
        results["bleu"] = bleu.compute(
            predictions=all_preds, references=[[r] for r in all_refs])

    return results, all_preds, all_labels if task_type == "classification" else all_refs
```


Building Demos with Gradio

Quick and Beautiful Web Interfaces for ML Models

Gradio is the fastest way to turn a Python function into a shareable web application. It handles all the HTML/CSS/JS, leaving you to focus on the model. A Gradio demo can be launched locally in seconds and shared publicly via a temporary HuggingFace tunnel with `share=True`.



Gradio: Basic Classification Interface

```
import gradio as gr
from transformers import pipeline

# ——— 1. Basic Text Classification Demo ———
def classify_sentiment(text):
    """Wrapper function for Gradio — takes raw text, returns label probabilities."""
    classifier = pipeline("sentiment-analysis", model="./my-model")
    result = classifier(text)[0]
    # Return a dict so Gradio renders a Label component with confidence bars
    return {"label": result["label"], "score": result["score"]}

demo = gr.Interface(
    fn = classify_sentiment,
    inputs = gr.Textbox(
        label = "Enter text",
        lines = 3,
        placeholder = "Type here..."
    ),
    outputs = gr.Label(label="Sentiment"),
    title = "Sentiment Analysis Demo",
    description = "Fine-tuned BERT for sentiment analysis",
    examples = [
        ["This movie was absolutely fantastic!"],
        ["Terrible waste of time and money."],
        ["It was okay, nothing special."],
    ],
)

# Local launch — set share=True for a public link (valid for 72 hours)
demo.launch(share=False)
```

Advanced Gradio: Chat Interface for LLMs

```
import gradio as gr
from transformers import pipeline

# ——— 2. Chat-Style Demo for Instruction-Tuned Models ———
generator = pipeline(
    "text-generation",
    model = "./my-instruction-model",
    tokenizer = "./my-instruction-model",
)

def chat_response(message, history):
    """
    Generate a response for the Gradio ChatInterface.

    Parameters
    -----
    message : str — the user's latest message
    history : list — list of [user, assistant] pairs (managed by Gradio)
    """
    # Format using the Alpaca instruction template
    prompt = (
        f"### Instruction:\nRespond to the user's message helpfully.\n\n"
        f"### Input:\n{message}\n\n### Response:\n"
    )
    response = generator(
        prompt,
        max_new_tokens = 200,
        do_sample = True,
        temperature = 0.7,
        top_p = 0.9,
    )[0]['generated_text']

    # Strip the prompt prefix — return only the model's response
    response = response.split("### Response:\n")[-1].strip()
    return response

# gr.ChatInterface handles conversation history automatically
demo = gr.ChatInterface(
    fn = chat_response,
    title = "Instruction-Tuned Chat Assistant",
    description = "Ask me anything! I can help with questions, tasks, and more.",
    examples = [
        "What is machine learning?",
        "Write a Python function to sort a list.",
        "Summarise: AI is transforming industries...",
    ],
    theme = "soft",
)
demo.launch()
```

Multi-Tab Gradio Application

```
import gradio as gr

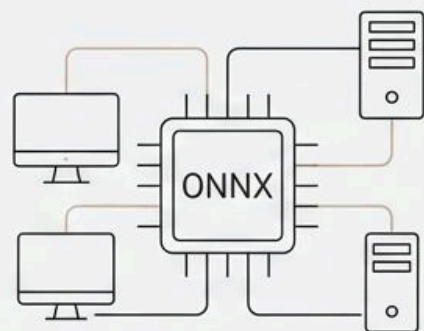
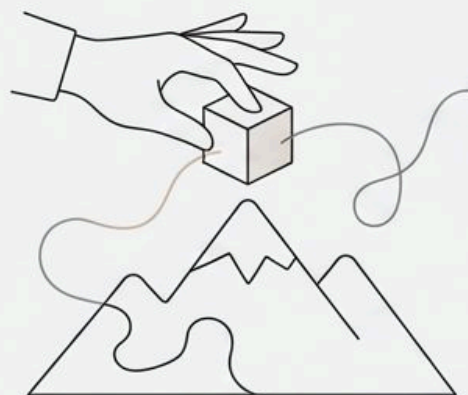
def create_multi_tab_demo():
    """Create a multi-functional Gradio app with separate tabs per capability."""
    with gr.Blocks(title="My Fine-Tuned Models", theme="soft") as demo:
        gr.Markdown("#
```

Model Export for Production

Choosing the Right Deployment Path for Your Use Case

HuggingFace Hub

Easiest. Push to hub, use Inference API.



ONNX Runtime

Cross-platform. Fast CPU/GPU inference.

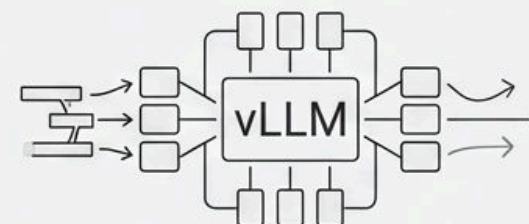
TGI

Production-grade Docker serving.



vLLM

High-throughput batch inference.



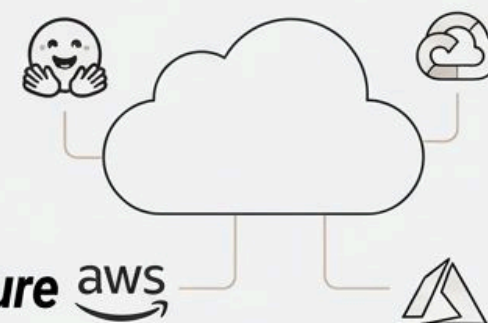
GGUF / llama.cpp

Local deployment. Ollama, LM Studio.



Cloud Services

HF Endpoints, AWS SageMaker, GCP, Azure



The right deployment path depends on your latency requirements, available infrastructure, and whether you need real-time or batch inference. For most teams, the recommended progression is: HuggingFace Hub → Gradio Space → TGI or vLLM for production load.

ONNX Export Example

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer
import torch
import onnx
import onnxruntime as ort
import numpy as np
```

```
# ——— Load fine-tuned model
```

```
model = AutoModelForSequenceClassification.from_pretrained("./my-model")
tokenizer = AutoTokenizer.from_pretrained("./my-model")
model.eval()
```

```
# ——— Create a dummy input for ONNX tracing
```

```
dummy_input = tokenizer(
    "Sample text for tracing",
    return_tensors = "pt",
    padding = True,
    truncation = True,
    max_length = 128,
)
```

```
# ——— Export to ONNX
```

```
torch.onnx.export(
    model,
    (dummy_input['input_ids'], dummy_input['attention_mask']),
    "model.onnx",
    input_names = ['input_ids', 'attention_mask'],
    output_names = ['logits'],
    dynamic_axes = {
        'input_ids': {0: 'batch_size', 1: 'sequence_length'},
        'attention_mask': {0: 'batch_size', 1: 'sequence_length'},
        'logits': {0: 'batch_size'},
    },
    opset_version = 14,
)
```

```
# ——— Verify model integrity
```

```
onnx_model = onnx.load("model.onnx")
onnx.checker.check_model(onnx_model)
print("Model exported successfully")
```

HuggingFace Spaces Deployment

——— app.py ——— (the entry point for your HuggingFace Space) ———

```
import gradio as gr
from transformers import pipeline
import torch

# Load model from Hub (or local path)
model_path = "your-username/your-model" # Public Hub model ID
classifier = pipeline(
    "text-classification",
    model=model_path,
    device = 0 if torch.cuda.is_available() else -1,
)

def predict(text):
    result = classifier(text)[0]
    return {result['label']: result['score']}

demo = gr.Interface(
    fn=predict,
    inputs=gr.Textbox(label="Text", lines=3),
    outputs=gr.Label(label="Prediction"),
    title="My Fine-Tuned Classifier",
    description="A fine-tuned model for text classification",
)
demo.launch()
```

——— requirements.txt

```
# transformers>=4.35.0
# torch>=2.0.0
# gradio>=4.0.0
# datasets>=2.14.0
```

——— Deployment steps

```
# 1. Go to https://huggingface.co/spaces and create a new Space
# 2. Select "Gradio" as the SDK
# 3. Push your code via Git:
```

```
# git clone https://huggingface.co/spaces/username/space-name
# cp app.py requirements.txt space-name/
# cd space-name
# git add .
# git commit -m "Add model demo"
# git push
```

```
# The Space will automatically install requirements and launch your demo.
# Free CPU Spaces are available; upgrade to GPU for faster inference.
```


Model Selection Framework

Choosing the Right Architecture for Your Task

START: DECISION TREE

(1) UNDERSTANDING TASKS



Encoder Models
(for speed, context, quality)

- Speed: **ModernBERT-base**
- Lightweight: **DistilBERT**
- Quality: **ModernBERT-large, DeBERTa**

8192

**ModernBERT
(Long Context)**

(2) GENERATION TASKS

under 4GB VRAM
SmolLM2-360M
or
Qwen2.5-0.5B

4-8GB VRAM
SmolLM2-1.7B
or **Phi-2**

8-16GB VRAM
Phi-3-mini or
Llama-3.2-3B

16GB+ VRAM
Llama-3.1-8B
or **Mistral-7B**

(3) SEQ2SEQ TASKS



Encoder-Decoder Models

- Lightweight: **t5-small, bart-tiny**
- Balanced: **t5-base, bart-base**
- Quality: **t5-large, bart-large**

When to Fine-Tune vs Prompt Engineer

A Practical Decision Framework

FINE-TUNE WHEN...

- You have 50+ high-quality labelled examples
Below this threshold, few-shot prompting typically outperforms fine-tuning due to limited signal.
- The task is specific or domain-specific
Legal, medical, or proprietary terminology that large APIs were never trained on.
- You need consistent, reliable outputs
Fine-tuned models produce more predictable output formats than prompted models, which can drift across versions.
- Latency is critical
A fine-tuned small model eliminates the overhead of long system prompts and can be deployed closer to users.

PROMPT ENGINEER WHEN...

- Few-shot or zero-shot is sufficient
Modern frontier models achieve strong performance on many tasks without any training.
- Rapid prototyping is needed
Prompt engineering delivers results in hours; fine-tuning pipelines take days to set up correctly.
- The task changes frequently
When requirements evolve weekly, retraining is impractical – update your prompt instead.
- No training infrastructure is available
API-only environments (e.g. OpenAI, Anthropic) preclude fine-tuning in many enterprise settings.

📌 **Recommended Hybrid Approach:** Start with prompt engineering on a large model → collect successful prompt-response pairs → fine-tune a smaller model on those pairs → deploy the smaller, faster, cheaper model in production.

Best Practices Checklist

PRE-TRAINING

- ☐ Define clear evaluation metrics before starting — avoid metric shopping post-hoc
- ☐ Collect and clean training data thoroughly — garbage in, garbage out
- ☐ Check for data quality issues: duplicates, label errors, demographic biases
- ☐ Create proper train / validation / test splits before any exploratory analysis
- ☐ Analyse class balance; address imbalance with oversampling or class weights
- ☐ Start with a small data subset for rapid experimentation before full runs

DURING TRAINING

- ☐ Monitor training and validation loss curves in real time (use W&B or TensorBoard)
- ☐ Watch for overfitting: growing gap between train and val metrics is a red flag
- ☐ Use gradient clipping (`max_grad_norm=1.0`) for numerical stability
- ☐ Log all hyperparameters for full reproducibility
- ☐ Save checkpoints regularly; never rely on a single final checkpoint

POST-TRAINING

- ☐ Evaluate on the held-out test set only once — never tune on it
- ☐ Perform error analysis on misclassified or poorly generated examples
- ☐ Test with adversarial inputs and edge cases before production deployment
- ☐ Document model limitations, known failure modes, and training data biases
- ☐ Create a model card with intended use, out-of-scope uses, and evaluation results
- ☐ Version your models — never overwrite a checkpoint that is live in production

DEPLOYMENT

- ☐ Use batch inference where possible — 10–30x throughput improvement over single-item
- ☐ Apply quantisation (8-bit or 4-bit) for CPU/edge deployments
- ☐ Cache predictions for repeated queries using `lru_cache` or Redis
- ☐ Load the pipeline once at startup; never re-instantiate per request

Performance Optimisation for Production

```
import torch
from transformers import pipeline, BitsAndBytesConfig, AutoModel
from functools import lru_cache

# — 1. Batch Inference — the single biggest win —
# WRONG: one-by-one inference (slow)
for text in texts:
    result = classifier(text)

# RIGHT: batch inference (10–30× faster depending on GPU)
results = classifier(texts, batch_size=32)

# — 2. Model Quantisation for Deployment —
bnb_config = BitsAndBytesConfig(
    load_in_8bit=True, # ~2× smaller than fp16, minimal quality loss
    # load_in_4bit=True, # ~4× smaller, small quality trade-off
)
model = AutoModel.from_pretrained(
    "my-model",
    quantization_config=bnb_config
)

# — 3. ONNX Runtime for CPU Inference —
# After ONNX export (see earlier slide), use onnxruntime for 2–3× faster CPU inference.
# session = ort.InferenceSession("model.onnx", providers=["CPUExecutionProvider"])

# — 4. Caching for Repeated Queries —
@lru_cache(maxsize=1000)
def cached_prediction(text):
    """Cache predictions for identical inputs — useful for FAQ-style apps."""
    return classifier(text)

# — 5. Use Pipeline Efficiently — load once, reuse everywhere —
# Load at module level (not inside a request handler)
classifier = pipeline("text-classification", model="my-model")

# Then simply call it in each request:
def handle_request(text: str) -> dict:
    return classifier(text)[0] # Reuses the same model object — no reload overhead
```

Lab: Gradio Demo and Model Sharing

30-Minute Deployment Walkthrough

01

Build a Multi-Functional Gradio Interface

Create a polished `gr.Blocks` app with generation controls, advanced settings, and example prompts.

03

Test the Demo Locally

Launch on port 7860, verify all event handlers work, and test edge cases including empty inputs.

02

Add Examples and Documentation

Populate the `gr.Examples` component and write clear Markdown descriptions for each tab.

04

Prepare for HuggingFace Spaces

Write `requirements.txt`, create a model card README, and push to the Hub using the HF API or Git.

Lab Exercise 1: Build Gradio Demo

```
print("=" * 60)
print("EXERCISE 1: Build Gradio Demo")
print("=" * 60)

import gradio as gr
from transformers import pipeline
import torch

# ——— Load fine-tuned model

model_path = "./fine-tuned-instruction-model"

try:
    from unsloth import FastLanguageModel
    model, tokenizer = FastLanguageModel.from_pretrained(
        model_path,
        max_seq_length = 1024,
        load_in_4bit = True,
    )
    FastLanguageModel.for_inference(model)

    def generate_response(prompt, max_tokens, temperature, top_p):
        formatted_prompt = f"### Instruction:\n{prompt}\n\n### Response:\n"
        inputs = tokenizer(formatted_prompt, return_tensors="pt").to(model.device)
        with torch.no_grad():
            outputs = model.generate(
                **inputs,
                max_new_tokens = max_tokens,
                do_sample = True if temperature > 0 else False,
                temperature = temperature,
                top_p = top_p,
                pad_token_id = tokenizer.pad_token_id,
            )
            response = tokenizer.decode(outputs[0], skip_special_tokens=True)
            response = response.split("### Response:\n")[-1].strip()
        return response

except Exception as e:
    print(f"Couldn't load fine-tuned model: {e}")
    print("Falling back to GPT-2...")
    generator = pipeline("text-generation", model="gpt2",
        device=0 if torch.cuda.is_available() else -1)

    def generate_response(prompt, max_tokens, temperature, top_p):
        result = generator(
            prompt,
            max_new_tokens = max_tokens,
            do_sample = True if temperature > 0 else False,
            temperature = temperature,
            top_p = top_p,
        )[0]['generated_text']
        return result

# ——— Build Gradio Blocks interface ———
with gr.Blocks(
    title = "Fine-Tuned Model Demo",
    theme = gr.themes.Soft(),
    css = "footer {display: none !important}"
) as demo:
    gr.Markdown("""
#
```

Lab Exercise 2: Model Sharing to HuggingFace Hub

```
print("\n" + "=" * 60)
print("EXERCISE 2: Share Model to HuggingFace Hub")
print("=" * 60)

# —— Login

# from huggingface_hub import notebook_login
# notebook_login() # Interactive login widget in Jupyter
# — or —
# huggingface-cli login # CLI login

# —— Write a model card (README.md) ——
model_card = """---
language: en
license: mit
tags:
- fine-tuned
- qlora
- instruction-following
- unsloth
base_model: HuggingFaceTB/SmolLM2-360M
---

# Instruction-Tuned SmolLM2-360M

## Model Description
Fine-tuned version of SmolLM2-360M using **QLoRA** and **Unsloth**.
Trained to follow instructions across sentiment analysis, summarisation, QA, and code generation.

## Intended Use
- Following general natural-language instructions
- Answering factual questions
- Basic text generation and classification tasks

## Training Details
| Setting | Value |
|-----|-----|
| Method | QLoRA (r=16) with Unsloth |
| Quantisation | 4-bit NF4 + double quantisation |
| Dataset | Custom multi-task instruction |
| Epochs | 3 |
| Hardware | Single GPU |
| Trainable parameters | ~2–3M (≈0.5–1% of total) |

## Limitations
- Small model size limits complex multi-step reasoning
- May produce factually incorrect information — always verify outputs
- Limited to English language
- Training data is small and partially synthetic

## How to Use
```python
from unsloth import FastLanguageModel
model, tokenizer = FastLanguageModel.from_pretrained(
 "your-username/your-model-name",
 load_in_4bit=True,
)
FastLanguageModel.for_inference(model)
```
"""

with open("./fine-tuned-instruction-model/README.md", "w") as f:
    f.write(model_card)
print("Model card created!")

# —— Push to Hub

print("\nTo push to HuggingFace Hub, choose one of:")

# Option A: Python API (recommended for programmatic workflows)
# from huggingface_hub import HfApi
# api = HfApi()
# api.upload_folder(
#     folder_path = "./fine-tuned-instruction-model",
#     repo_id = "your-username/your-model-name",
#     repo_type = "model",
# )

# Option B: Git (familiar workflow for teams already using Git LFS)
# cd ./fine-tuned-instruction-model
# git init
# git remote add origin https://huggingface.co/your-username/your-model-name
# git add .
# git commit -m "Upload fine-tuned model"
# git push origin main

# Option C: Unsloth one-liner (pushes adapters + merged model in one step)
# model.push_to_hub_merged(
#     "your-username/your-model-name",
#     tokenizer,
#     save_method="merged_16bit",
# )
```


Lab Exercise 3: Final Evaluation and Course Summary

```
print("\n" + "=" * 60)
print("EXERCISE 3: Final Evaluation and Summary")
print("=" * 60)

# ——— Project summary ———


---



print("\n" + "=" * 50)
print("COURSE PROJECT SUMMARY")
print("=" * 50)

project_summary = {
    "Model Architecture": "SmolLM2-360M (Transformer Decoder, 360M params)",
    "Fine-tuning Method": "QLoRA with Unsloth optimisations",
    "LoRA Configuration": "r=16, alpha=16, all attention + FFN projection layers",
    "Quantisation": "4-bit NF4 with Double Quantisation",
    "Training Data": "Custom multi-task instruction dataset (varied tasks)",
    "Training Duration": "~5–15 minutes on a single consumer GPU",
    "Peak Memory Usage": "~3–4 GB during training (vs ~8–10 GB standard)",
    "Trainable Parameters": "~2–3M (≈0.5–1% of total — the LoRA adapters only)",
}

for key, value in project_summary.items():
    print(f" {key}: {value}")

# ——— What we accomplished across the 5-day course ———


---



print("\n" + "=" * 50)
print("WHAT WE ACCOMPLISHED IN THIS COURSE")
print("=" * 50)

accomplishments = [
    "
```

Course Wrap-Up: Key Themes

The Transformers Fine-Tuning Mindset

1. Start Simple, Then Optimise

Pipeline API → Custom Models → Fine-Tuning → PEFT. Each step adds complexity — only move to the next when the previous step is insufficient for your requirements.

2. Right-Size Your Approach

A small fine-tuned model often outperforms a large model with poor prompting, at a fraction of the inference cost. Match model size to task complexity.

3. Memory Is the Constraint

LoRA, QLoRA, and quantisation are not compromises — they are the correct tools for single-GPU training. Unsloth makes them even faster without changing your results.

4. Evaluate Rigorously

Define metrics before training. Use both intrinsic (perplexity) and extrinsic (F1, BLEU, ROUGE) metrics. Error analysis reveals where to invest next.

5. Deploy Thoughtfully

Gradio for demos, ONNX/GGUF for production edge, TGI/vLLM for high-throughput serving. Document limitations and version every model you ship.

Final Challenge: Take-Home Projects

Apply What You've Learned — Choose One Project

1

Domain-Specific Classifier

Pick a domain (medical, legal, finance, or your own). Collect or create a labelled dataset of 50–500 examples. Fine-tune a BERT/DistilBERT model, evaluate with F1 and accuracy, and deploy with Gradio. **Bonus:** compare against a zero-shot GPT-4 baseline — quantify how much fine-tuning helps.

2

Custom Instruction Model

Create an instruction dataset for a specific use case (customer support, code review, study assistant). Fine-tune SmolLM2 or Phi with QLoRA and Unsloth. Test on diverse prompt phrasings not seen during training. **Bonus:** export to GGUF and run locally via Ollama — compare quality and latency.

3

Multi-Task Fine-Tuned Model

Combine 2–3 related tasks into one instruction-formatted dataset. Fine-tune using the instruction format and evaluate each task separately. Analyse task interference — does adding task B help or hurt task A? **Bonus:** implement task-specific LoRA adapters and compare vs shared adapters.

 Share your results on HuggingFace Spaces! Tag the model card with `huggingface-transformers-course` so the community can find your work.

Resources for Continued Learning

Advanced Topics to Explore Next

- RLHF & DPO

Reinforcement Learning from Human Feedback and Direct Preference Optimisation — the techniques behind ChatGPT-style alignment.

- Mixture of Experts (MoE)

Sparse routing architectures that activate only a subset of parameters per token — enabling very large models at lower inference cost.

- Speculative Decoding

Use a small draft model to propose tokens, then verify with the large model — achieving 2–3x inference speedup with identical outputs.

- Multimodal Models

Vision-language (Llama-3.2-Vision, Qwen2-VL) and speech models that process multiple modalities within a single transformer.

Tools to Explore

- **vLLM** — PagedAttention for high-throughput batch inference
- **Text Generation Inference (TGI)** — Production-grade Docker serving
- **LangChain / LlamaIndex** — LLM application frameworks (RAG, agents)
- **Axolotl** — Flexible fine-tuning framework with rich config support
- **llama.cpp / Ollama** — Local LLM inference on CPU and Apple Silicon

Learning Resources

- **HuggingFace NLP Course (free)**
- **Fast.ai Practical Deep Learning (free)**
- Stanford CS224N: NLP with Deep Learning
- Andrej Karpathy's "Let's build GPT" (YouTube)
- **Unsloth Documentation and Tutorials**

Communities

- HuggingFace Discord — **#fine-tuning** channel
- r/LocalLLaMA on Reddit — active community for local model deployment
- Unsloth Discord — framework-specific support and tutorials

Key Takeaways — Module 10



Model Evaluation

Use both intrinsic metrics (perplexity, cross-entropy loss) to monitor training health and extrinsic metrics (accuracy, F1, BLEU, ROUGE) to measure real-world task performance. Define your primary metric *before* training begins to avoid post-hoc metric shopping.



Gradio for Demos

Gradio's `gr.Interface` and `gr.Blocks` APIs enable polished, shareable web demos in under 50 lines of Python. `gr.ChatInterface` handles multi-turn conversation history automatically — ideal for instruction-tuned models.



Multiple Deployment Paths

Choose the right path for your needs: HuggingFace Hub and Spaces for rapid sharing, ONNX for cross-platform CPU inference, GGUF for local deployment via Ollama and llama.cpp, TGI or vLLM for production-grade high-throughput serving.



Model Selection Framework

Match architecture to task: encoder models (BERT family) for understanding tasks, decoder models (GPT family) for generation, encoder-decoder (T5/BART) for seq2seq. Within each family, let your VRAM budget and quality requirements determine the specific model size.



Fine-Tune vs Prompt Engineer

Prompt engineering for rapid prototyping and general tasks; fine-tuning when you have 50+ labelled examples, need consistent output formats, or require low-latency inference without long system prompts. The hybrid approach — prompt first, fine-tune later — is often optimal.

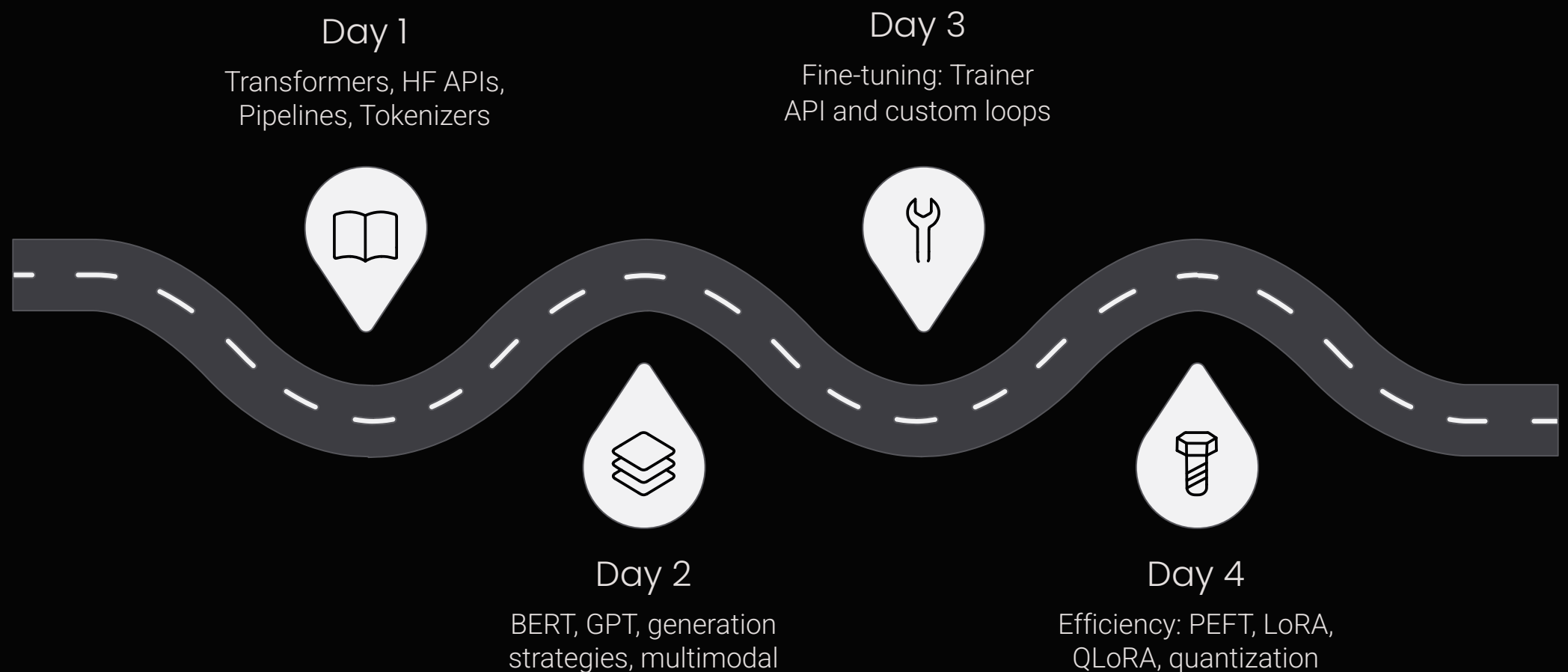


Best Practices

Evaluate rigorously on a held-out test set (only once!), perform error analysis on failures, document all model limitations and training data biases, create a model card, and version every model you deploy — never overwrite a checkpoint that is live in production.

Complete Course Summary

The Full Fine-Tuning Pipeline — Five Days in Review



Each day built directly on the previous: foundational API literacy enabled architectural understanding, which enabled full fine-tuning, which enabled efficient fine-tuning with PEFT, which enabled production-grade instruction tuning and deployment. The progression was intentional — every concept introduced was a prerequisite for the next.

Key Concept 1

Transformers are universal. The same attention mechanism underlies encoder, decoder, and encoder-decoder models — only the masking strategy and training objective differ.

Key Concept 2

PEFT democratises fine-tuning. LoRA and QLoRA reduce trainable parameters by 99% — making fine-tuning accessible on a single consumer GPU with 6–8 GB VRAM.

Key Concept 3

Instruction tuning unlocks generalisation. A model trained to follow instructions can handle unseen task variants at inference time — making it far more robust than task-specific models.

Key Concept 4

Deployment is part of the pipeline. A model that cannot be evaluated, shared, and served reliably has no real-world impact. Gradio, GGUF, and TGI complete the loop.

Skills You've Acquired

A Practitioner's Toolkit — Ready for Real Projects



Days of Training

Covering the complete fine-tuning lifecycle from raw API to production deployment



Model Families

Llama, Mistral, Phi, Gemma, Qwen, BERT, T5, SmolLM2, and more



Core Skills

From tokenisation and dataset construction to QLoRA, Unsloth, and Gradio deployment

→ Select appropriate models for specific tasks and hardware constraints

You can now read a model card, assess VRAM requirements, and choose the right architecture without trial and error.

→ Implement and optimise fine-tuning end-to-end

From raw dataset construction through QLoRA training with Unsloth to checkpoint management and evaluation.

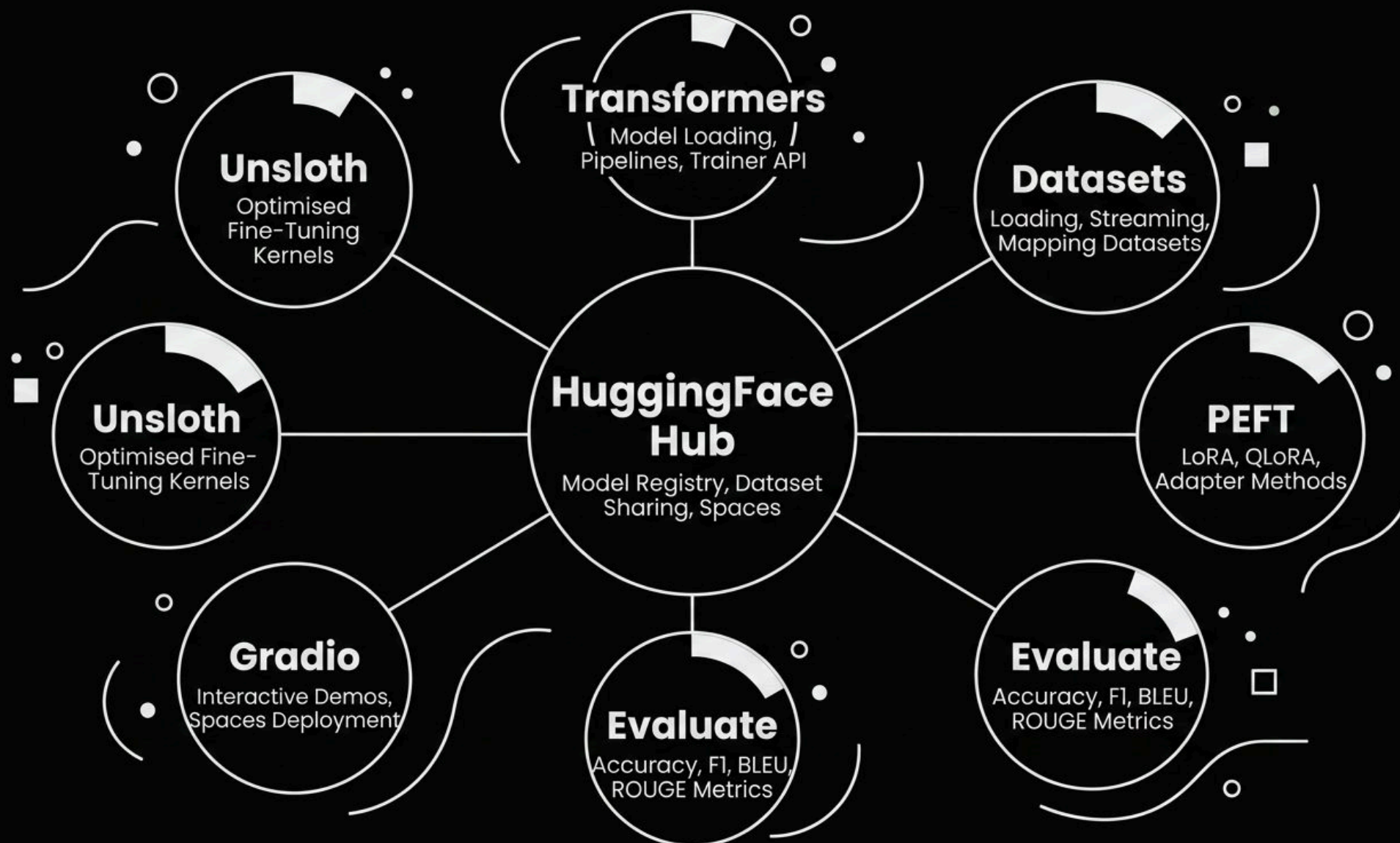
→ Deploy models as interactive applications

Gradio for demos, GGUF for local inference, ONNX for cross-platform production, and HuggingFace Spaces for sharing.

→ Evaluate systematically and iterate with confidence

Intrinsic and extrinsic metrics, error analysis, and the best-practices checklist give you a repeatable quality framework for every future project.

Quick Reference: HuggingFace Ecosystem Map



All six libraries are interoperable and share the same model hub as a central registry. A model trained with Transformers + PEFT + Unsloth can be evaluated with Evaluate, demoed with Gradio, and pushed to the Hub – all without format conversion or compatibility issues. This ecosystem cohesion is HuggingFace's greatest engineering achievement and the reason it has become the de facto standard for applied NLP.

Unsloth + HuggingFace: The Recommended Stack for 2025

Putting It All Together in One Workflow

```
"""
RECOMMENDED FINE-TUNING STACK FOR 2025
"""

# ----- Step 1: Install dependencies -----
# pip install unsloth datasets evaluate gradio huggingface_hub

# ----- Step 2: Load model with Unsloth -----
from unsloth import FastLanguageModel
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Llama-3.2-3B",
    max_seq_length = 2048,
    load_in_4bit = True,
)
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
"gate_proj", "up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    use_gradient_checkpointing = "unsloth",
)

# ----- Step 3: Prepare dataset -----

from datasets import load_dataset
dataset = load_dataset("your-org/your-instruction-dataset")

def format_prompt(example):
    return {"text": f"### Instruction:\n{example['instruction']}\n\n"
f"### Input:\n{example['input']}\n\n"
f"### Response:\n{example['output']}"}

dataset = dataset.map(format_prompt)

# ----- Step 4: Train -----

from transformers import TrainingArguments, Trainer
from unsloth import is_bfloat16_supported

trainer = Trainer(
    model = model,
    tokenizer = tokenizer,
    args = TrainingArguments(
        output_dir = "./my-model",
        num_train_epochs = 3,
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        learning_rate = 2e-4,
        bf16 = is_bfloat16_supported(),
        fp16 = not is_bfloat16_supported(),
        optim = "adamw_8bit",
        report_to = "none",
    ),
    train_dataset = dataset["train"],
)
trainer.train()

# ----- Step 5: Export to GGUF -----

model.save_pretrained_gguf("./my-model-gguf", tokenizer, quantization_method="q4_k_m")

# ----- Step 6: Push to Hub -----

model.push_to_hub_merged("your-username/my-model", tokenizer, save_method="merged_16bit")

# ----- Step 7: Deploy with Gradio -----
import gradio as gr
import torch

FastLanguageModel.for_inference(model)

def respond(prompt):
    inputs = tokenizer(f"### Instruction:\n{prompt}\n\n### Response:\n", return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=200, temperature=0.7, do_sample=True)
    return tokenizer.decode(outputs[0], skip_special_tokens=True).split("### Response:\n")[-1].strip()

gr.Interface(fn=respond, inputs="text", outputs="text", title="My Fine-Tuned Model").launch()
```

Thank You!

Congratulations on Completing the Course!

What You've Learned

- Transformer Architectures & HuggingFace APIs

Pipelines, tokenisers, model hubs, and the full ecosystem of interoperable libraries.

- Full Fine-Tuning, PEFT/LoRA, and QLoRA

From scratch full fine-tuning through parameter-efficient adapters to 4-bit quantised training on consumer hardware.

- Instruction Tuning and Domain Adaptation

Dataset formats, chat templates, continued pre-training, multi-task learning, and data augmentation strategies.

- Unsloth, Evaluation, and Deployment

2–5× faster training, rigorous metric-driven evaluation, and production-ready deployment with Gradio, ONNX, and GGUF.

Remember

Start Simple, Iterate Fast

The best model is the one you can actually train, evaluate, and deploy — not the theoretically optimal one.

Smaller + Fine-Tuned Beats Larger + Prompted

A well-tuned 360M model routinely outperforms a 7B model given a mediocre prompt on domain-specific tasks.

Share Your Work

The open-source NLP community grows stronger every time someone publishes a model, dataset, or demo. Be a contributor.

"Keep building! The best way to understand a model is to train one."

Additional Resources & Contact

Course Materials

- [GitHub Repository — Samples and Presentations](#)

Reach Out

- chandrashekar.info/contact
- [LinkedIn — Chandrashekar Babu](#)
- slashprog.com
- training@chandrashekar.info

Communities

- [HuggingFace Hub](#)
- [HuggingFace Forums](#)
- [Unsloth Discord](#)

Official Documentation

- [HuggingFace Transformers](#)
- [PEFT Library](#)
- [Unsloth Documentation](#)
- [Gradio Documentation](#)

Key GitHub Repositories

- [unslothai/unsloth](https://github.com/unslothai/unsloth)
- [huggingface/transformers](https://github.com/huggingface/transformers)
- [huggingface/peft](https://github.com/huggingface/peft)

Key Papers

- [InstructGPT — Training LMs to Follow Instructions](#)
- [Self-Instruct — Aligning LMs with Self-Generated Instructions](#)

 Thank you for your focus, curiosity, and engagement across all five days. The community gets stronger every time a practitioner like you ships a model. **Keep building!** 